FIAP ON — Navegando pelo mundo DevOps

Projeto Cidades ESG Inteligentes

EcoSmart Porto Verde — API Java Spring Boot + MongoDB com pipeline CI/CD, Docker e Docker Compose

Integrantes:

[Nome completo e RM de cada integrante — preencher]

Repositório: [link] | Pipeline: [link]

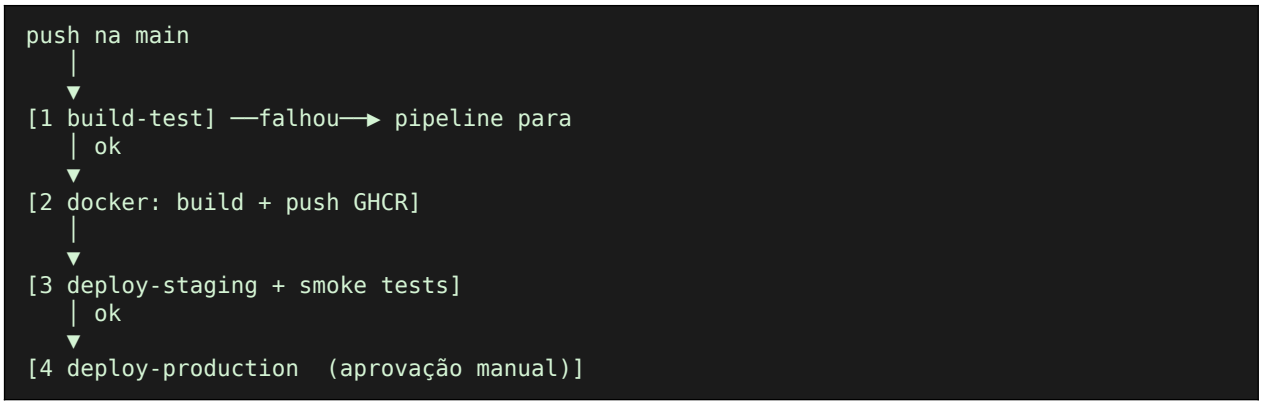
1. Descrição do pipeline

Ferramenta utilizada: GitHub Actions. O workflow está em .github/workflows/ci-cd.yml e é disparado em push e pull request na branch main (e manualmente via workflow_dispatch). Os deploys só acontecem em push na main.

Etapa (job)	Lógica
1. build-test	Configura Java 21 com cache do Maven e executa mvn -B verify: compila o projeto e roda os 7 testes automatizados (JUnit 5 + Mockito + MockMvc, sem depender de banco). Publica o relatório de testes e o .jar como artefatos.
2. docker	Depende do job 1. Faz o build da imagem com Buildx (cache gha). Em push na main, autentica no GHCR com o GITHUB_TOKEN e publica ghcr.io/<owner>/<repo>:<sha-curto> e :latest. Em pull request, apenas valida o build.
3. deploy-staging	Depende do job 2. Usa o environment 'staging': executa scripts/deploy.sh staging (docker compose pull/up com env/staging.env, porta 8081), aguarda o health check e roda smoke tests (/actuator/health, /api/ambiente, /api/sensores).
4. deploy-production	Depende do staging. Usa o environment 'production' (aprovação manual via Required reviewers): mesmo script com env/production.env (porta 8082, banco e projeto Compose próprios).

Lógica de proteção: falha em build ou testes interrompe o pipeline antes de qualquer imagem ser publicada; produção só recebe a mesma imagem (mesmo SHA) que já passou no staging; as senhas vêm de GitHub Secrets por ambiente e nunca ficam no repositório.

Fluxo do pipeline



2. Docker

Arquitetura



```
          ▼
   porta APP_PORT (8080 local / 8081 staging / 8082 prod)
```

Imagem criada

- Dockerfile multi-stage: estágio build (maven:3.9-eclipse-temurin-21) e estágio final eclipse-temurin:21-jre-alpine.
- Camada de dependências em cache (pom.xml copiado antes do código), usuário não-root, HEALTHCHECK via /actuator/health.
- Imagem publicada no GHCR: ghcr.io/<owner>/<repo>:<sha> e :latest.

Orquestração (docker-compose.yml)

- Serviços: mongo (mongo:7) e app (Spring Boot).
- Volumes: mongo-data (dados do banco) e app-logs (logs da aplicação).
- Rede: backend (bridge); o MongoDB não é exposto ao host.
- Variáveis de ambiente: via .env / env/*.env (MONGO_USER, MONGO_PASSWORD, MONGO_DB, APP_ENV, APP_PORT, SEED_DATA).
- depends_on com condition: service_healthy: a API só inicia após o Mongo responder ao ping.

Comandos usados

```
cp .env.example .env
docker compose up --build -d
docker compose ps
docker compose logs -f app
curl http://localhost:8080/actuator/health
curl http://localhost:8080/api/ambiente
docker compose down        # mantém volumes
docker compose down -v     # remove volumes
```

3. Prints do pipeline rodando

[INSERIR PRINT] Visão geral do workflow com os 4 jobs (GitHub → Actions)

[INSERIR PRINT] Job build-test: log do 'mvn -B verify' com testes passando (Tests run: 7, Failures: 0)

[INSERIR PRINT] Job docker: imagem enviada ao GHCR (aba Packages)

[INSERIR PRINT] Job deploy-staging e deploy-production: logs do deploy.sh com 'Deploy ... concluido com sucesso'

4. Prints dos ambientes staging e produção

[INSERIR PRINT] Staging: curl / navegador em /api/ambiente retornando "ambiente":"staging" e /api/sensores com dados

[INSERIR PRINT] Produção: /api/ambiente retornando "ambiente":"production" e /actuator/health UP

[INSERIR PRINT] docker compose ps mostrando mongo e app saudáveis (local ou no runner)

5. Desafios encontrados e como foram resolvidos

Os pontos abaixo são os desafios de projeto tratados nesta solução. Ajuste/complemente com o que o grupo realmente enfrentou ao executar o pipeline.

Desafio	Solução
A API iniciava antes do MongoDB estar pronto e falhava ao conectar.	healthcheck no serviço mongo (ping) e depends_on com condition: service_healthy no serviço app.
Testes dependiam de banco, tornando o pipeline lento e instável.	Testes de controller com MockMvc standalone e repositórios simulados (Mockito); o banco real é validado nos smoke tests do deploy.
Duas versões (staging e produção) no mesmo host.	Um arquivo env/<ambiente>.env por ambiente, com COMPOSE_PROJECT_NAME, porta e banco próprios, evitando conflito de containers, volumes e portas.
Segredos (senha do banco) não podem ir para o Git.	Secrets por environment do GitHub; os arquivos versionados têm apenas variáveis não sensíveis; .env no .gitignore e .env.example como modelo.
GHCR rejeita nomes de imagem com letra maiúscula.	O nome é convertido para minúsculas no pipeline (${GITHUB_REPOSITORY,,}).
Garantir que produção rode exatamente o que foi testado.	A imagem é construída uma vez, versionada pelo SHA do commit, e o mesmo SHA é usado em staging e produção.
Imagem grande e builds lentos.	Build multi-stage com JRE alpine e cache de dependências Maven e de camadas Docker (cache gha).

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑️
Dockerfile funcional	☑️
docker-compose.yml ou arquivos Kubernetes	☑️
Pipeline com etapas de build, teste e deploy	☑️
README.md com instruções e prints	☐ marcar após inserir os prints
Documentação técnica com evidências (PDF ou PPT)	☐ marcar após inserir os prints
Deploy realizado nos ambientes staging e produção	☐ marcar após o pipeline rodar no GitHub